DOI:10.1145/3618117

To view the accompanying Technical Perspective, visit doi.acm.org/10.1145/3657629

OpenVPN Is Open to VPN Fingerprinting

By Diwen Xue, Reethika Ramesh, Arham Jain, Michaelis Kallitsis, J. Alex Halderman, Jedidiah R. Crandall, and Roya Ensafi

Abstract

VPN adoption has seen steady growth over the past decade due to increased public awareness of privacy and surveillance threats. In response, certain governments are attempting to restrict VPN access by identifying connections using "dual use" DPI technology. To investigate the potential for VPN blocking, we develop mechanisms for accurately fingerprinting connections using OpenVPN, the most popular protocol for commercial VPN services. We identify three fingerprints based on protocol features such as byte pattern, packet size, and server response. Playing the role of an attacker who controls the network, we design a two-phase framework that performs passive fingerprinting and active probing in sequence. We evaluate our framework in partnership with a million-user ISP and find that we identify more than 85% of OpenVPN flows with only negligible false positives, suggesting that OpenVPN-based services can be effectively blocked with little collateral damage. Although some commercial VPNs implement countermeasures to avoid detection, our framework successfully identified connections to 34 out of 41 "obfuscated" VPN configurations. We discuss the implications of the VPN fingerprintability for different threat models and propose short-term defenses. In the longer term, we urge commercial VPN providers to be more transparent about their obfuscation approaches and to adopt more principled detection countermeasures, such as those developed in censorship circumvention research.

1. INTRODUCTION

ISPs, advertisers, and national governments are increasingly disrupting, manipulating, and monitoring Internet traffic. As a result, virtual private network (VPN) adoption has been growing rapidly, not only among activists and journalists with heightened threat models but also among average users, who employ VPNs for reasons ranging from protecting their privacy on untrusted networks to circumventing censorship.

In response to the growing popularity of VPNs, numerous ISPs and governments are now seeking to track or block VPN traffic in order to maintain visibility and control over the traffic within their jurisdictions. Binxing Fang, the architect of the Great Firewall of China (GFW), characterized the situation as an "eternal war" between the firewall and VPNs. Consequently, the country has mandated ISPs to identify and block personal VPN usage.[18,19] Similarly, both Russia and India have recently proposed bans on VPN services within their borders, citing national cybersecurity concerns.[13,17] Commercial ISPs, too, have shown an interest in tracking VPN connections. For example, in 2021, a large ISP in South Africa, Rain, Ltd., reduced VPN connection speeds by more than 90%, a measure intended to enforce quality-of-service limitations in its data plans.[20] These motivations are compounded by the accessibility of advanced technologies such as carrier-grade deep packet inspection (DPI) to even smaller ISPs and censors. Such technologies enable the application of more sophisticated detections based on protocol semantics.[12,14]

In this paper, we explore the implications of DPI for VPN detection and blocking by studying the fingerprintability of OpenVPN (the most popular protocol for commercial VPN services) from the perspective of an adversarial ISP. We seek to answer two research questions: *(1) Can ISPs and governments identify traffic flows as OpenVPN connections in real time*? and *(2) Can they do so at scale without incurring significant collateral damage from false positives?* Answering these questions requires more than just identifying fingerprinting vulnerabilities; although challenging, we need to demonstrate practical exploits under the constraints of how ISPs and nation-state censors operate in the real world.

We build a detection framework inspired by the architecture of the Great Firewall,[2,4,23] consisting of *filter* and *prober* components. A filter performs passive filtering over passing network traffic in real time, exploiting protocol quirks we identified in OpenVPN's handshake stage. After a flow is flagged by a filter, the destination address is passed to a prober that performs active probing as confirmation. By sending probes carefully designed to elicit protocol-specific behaviors, the prober is able to identify an OpenVPN server using side channels even if the server enables OpenVPN's optional defense against active probing. Our two-phase framework is capable of processing ISP-scale traffic at line-speed with an extremely low false-positive rate.

In addition to core or "vanilla" OpenVPN, we also include commercial "obfuscated" VPN services in this study. In response to increasing interference from ISPs and censors, obfuscated VPN services have started to gain traction, especially from users in countries with heavy censorship or laws against the personal usage of VPNs. Obfuscated VPN services, whose operators often tout them as "invisible" and "unblockable," typically use OpenVPN as the underlying protocol with an obfuscation layer to avoid detection.

The original version of this paper was published in *Proceedings of the 31st USENIX Security Symposium*, (2022).

Partnering with Merit (a mid-size regional ISP that serves a population of 1 million users), we deploy our framework at a monitor server that observes 20Gbps of ingress and egress traffic mirrored from a major Merit point of presence. In our tests, we are able to identify 1,718 out of 2,000 flows originating from a control client machine residing within the network, corresponding to 39 out of 40 unique "vanilla" OpenVPN configurations. More strikingly, we also successfully identify more than two-thirds of obfuscated OpenVPN flows. Eight of the top 10 providers offer obfuscated services, yet all of them are flagged by our filter. Despite providers' lofty unobservability claims (such as "... even your Internet provider can't tell that you're using a VPN"), we find most implementations of obfuscated services resemble OpenVPN masked with the simple XORPatch,[1] which is easily fingerprintable. Lack of random padding at the obfuscation layer and co-location with vanilla OpenVPN servers also make the obfuscated services more vulnerable to detection.

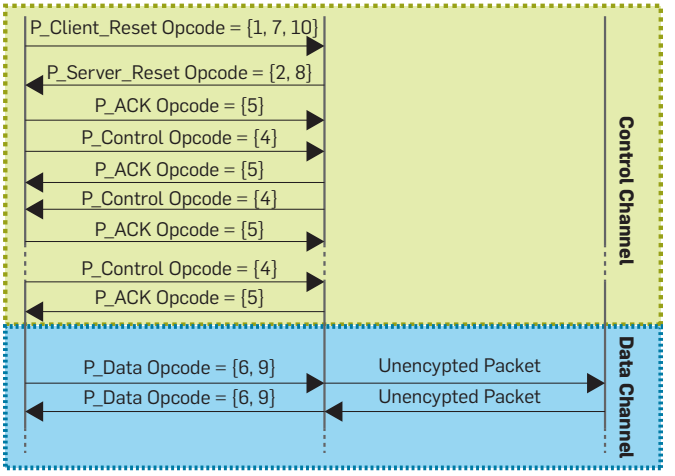
In a typical day, our single-server setup analyzes 15TB of traffic and 2 billion flows. Over an eight-day evaluation, our framework flagged 3,638 flows as OpenVPN connections. Among these, we are able to find evidence that supports our detection results for 3,245 flows, suggesting an upper-bound false-positive rate three orders of magnitude lower than previous machine learning (ML)-based approaches.[3,6]

We conclude that tracking and blocking the use of OpenVPN, even with most current obfuscation methods, is straightforward and within the reach of any ISP or network operator, as well as nation-state adversaries. Unlike circumvention systems, such as Tor pluggable transports,[10] which employ sophisticated strategies to avoid detection, robust obfuscation techniques have been conspicuously absent from OpenVPN and the broader VPN ecosystem. For average users, this means they may face blocking or throttling from ISPs, but for high-profile, sensitive users, this fingerprintability may lead to follow-up attacks that aim to compromise the security of OpenVPN tunnels.[15] We warn users with heightened threat models *not* to expect that their VPN usage will be unobservable, even when connected to "obfuscated" services. While we propose several short-term defenses for the fingerprinting exploits described in this paper, we fear that, in the long term, a cat-and-mouse game similar to the one between the Great Firewall and Tor is imminent in the VPN ecosystem as well. We implore VPN developers and providers to develop, standardize, and adopt robust, well-validated obfuscation strategies and to adapt them as the threats posed by adversaries continue to evolve.

2. OPENVPN, VPN DETECTION, OBFUSCATED VPN

VPN tools create private networks across the public Internet through encrypted tunneling. OpenVPN, first released in 2002, aims to create a tunneling protocol focusing on security while also being free and fast over the standard TCP and UDP. For each OpenVPN session, two separate channels are used for key exchange and data transfer, both sharing a single multiplexed TCP/UDP stream. In the control channel, the client and server engage in a TLS-style exchange of key materials. As TLS is designed to operate over a reliable transport, OpenVPN provides its control channel with a sequential, reliable layer based on an explicit acknowledgement and re-transmission mechanism. The negotiated key from the control channel will be used to encrypt packets transferred in the data channel, which does not provide any reliability guarantee. Figure 1 shows an initialization packet sequence leading to a fully encrypted data channel.

Figure 1. OpenVPN Session Establishment. Packets are annotated with message types and the associated opcodes.



The ongoing arms race between the GFW and Tor is most representative of the conflict between censorship and surveillance and circumvention tools.[2,4,23] Censors started by blocking Tor's website and public relays, which Tor responded to by deploying website mirrors and private, unpublished bridges. Next, censors moved to blocking with DPI by fingerprinting Tor's TLS handshake, for example, cipher suites. Tor used pluggable transports (PT) obfuscators, such as Obfsproxy and meek,[10] to mask the handshake. In response, censors deployed active probing to complement DPI-based fingerprinting to detect Tor and certain obfuscators.

There is limited previous work on VPN traffic detection. Hoogstraaten[8] explored server-side VPN detection methods, ranging from using existing information databases (for example, WHOIS) to fingerprinting TCP options. Webb et al.[22] proposed detecting proxies and VPNs based on timing and latency. Their approach relied on the hypothesis that when a service is accessed through a proxy, the RTT measurement will be different from direct connections. Another class of previous work uses ML models to passively detect VPN traffic,[3,6,7] leveraging flow-level statistics such as connection duration and packet interval. Most of this work uses the same synthetic dataset[7]—which contains a balanced mixture of VPN and non-VPN traffic—to train and test a variety of classifiers in an offline, lab-setting.

Various traffic obfuscation techniques have been examined in previous work. Wang et al. examined the detectability of Obfsproxy, FTE, and meek.[21] Using attacks based on protocol semantics, packet entropy, and timing-related features, they concluded that a determined censor could

detect all three obfuscators reliably. Houmansadr et al. demonstrated that popular mimicry-based obfuscation tools failed to achieve unobservability because seamlessly simulating another protocol is extremely challenging.[9] Previous studies have suggested censors can use active probing to detect proxies that obfuscate traffic.[2,4,23] In response, "probe-resistant" proxies were developed, which remain silent when being probed by an unauthenticated adversary.[5]

There is a marked demand for an emerging class of services called "stealth" or "obfuscated" VPN, especially from users in countries with heavy censorship or laws against personal VPN usage. Most obfuscated VPN services use OpenVPN as the underlying protocol for security and routing, with an obfuscation layer overlaid to avoid detection. The absence of a standardized obfuscation solution within OpenVPN has led to a plethora of obfuscators implemented by different VPN providers, who often claim that their obfuscated services can remain undetected by ISPs and censors alike. For example, TorGuard introduces their obfuscated VPN service as "Engineered from the ground up to be impossible to detect". BolehVPN claims that their VPN obfuscation "...keeps you out of trouble, even in China".

Common obfuscation strategies adopted by commercial VPNs include employing XOR-based scramblers, wrapping OpenVPN inside encrypted tunnels, or using proprietary protocols. The XOR patch, originally developed by Clayface as a patch for vanilla OpenVPN, scrambles a packet by either xor-ing bytes with a pre-shared key, reversing the order of the bytes, xor-ing each byte with its position, or a combination of these steps.[1] Other VPN services wrap OpenVPN traffic inside encrypted tunnels to prevent DPI fingerprinting. Some of the adopted obfuscation tunnels are Obfsproxy (obfs{2/3/4}), Stunnel, Websocket Tunnel, and encrypted proxies (shadowsocks, V2Ray). Finally, a few VPN providers have developed proprietary obfuscated protocols, some of which are built on top of OpenVPN with a proprietary obfuscation layer added.

To the best of our knowledge, we are the first to explore the fingerprintability of commercial or obfuscated OpenVPN services on real traffic. Our unique study highlights the practicality of such fingerprinting, with profound implications on end users expecting certain privacy and anonymity guarantees from using these services.

3. CHALLENGES IN REAL-WORLD VPN DETECTION

Effective investigation of fingerprintability requires incorporating perspectives of how ISPs and censors operate in practice. It is not enough to simply identify fingerprinting vulnerabilities; we need to demonstrate realistic exploits to illustrate the practicality of exploiting the vulnerability, while taking into consideration the ISP and censors' capabilities and constraints.[16] For instance, previous academic works considered using flow-level features to train ML classifiers for VPN detection.[3,6,7] Yet, it remains unclear how practical these detection approaches are for ISPs and censors, and we know of no rigorous studies that examine real-world deployment of an ML-based censorship system.[16] Furthermore, previous works test on the `ISCXVPN2016` dataset[7] with balanced OpenVPN and non-VPN traffic. However, we note that due to the low base rate of VPN traffic in the wild, even the best-performing ML system has false-positive rates that can be economically impractical for real-world censors sensitive to collateral damage.[21]

However, investigations adopting the viewpoint of ISPs and censors can be challenging. First, such investigation requires collaboration with real-world ISPs and access to their network traffic. We need to install monitors inside an ISP's network, while ensuring our analysis will not affect the ISP's normal routing operations. Furthermore, analyzing traffic from real users raises ethical concerns. Processing raw network data may violate the privacy of users, in particular VPN users who often have a heightened threat model. Finally, deploying a system that performs ad hoc traffic analysis in real time poses significant engineering challenges. We need to ensure the entire analysis framework (including processing and logging) keeps pace with the packet arrival rate and must consider the effect of potential asymmetric routing or packet loss on the analysis and results.

4. ADVERSARY MODEL AND DEPLOYMENT

We assume a realistic censor (ISP) capability model based on knowledge from previous measurement studies on the arms race between censors and circumventors.[2,4,16,23] We outline a censor-controlled on-path filter that passively observes and examines passing network traffic. The filter is stateful, but has limited resources and can maintain a limited amount of per-connection states for a short time. The filter is also constrained by long-term data storage and computational resources. In addition to filters installed inside the monitored networks, we assume the censor also operates measurement machines that can send protocol-specific probes to further confirm the detection result. Such two-phase systems have already been adopted by real-world censors, such as the GFW against Tor and Shadowsocks.[2,23] Finally, we expect the censor is familiar with the protocol of interest and has access to the different obfuscators deployed by VPN providers (for example, as a paid customer). We emphasize that this threat model corresponds to censor's capabilities as observed in practice *today* rather than future capabilities.

To investigate the fingerprintability of OpenVPN and existing obfuscated solutions, we set up a two-phase detection framework to answer our key questions: 1) Are real-world censors *capable* of performing such detection? and 2) Is it is economical to do this *at scale*? Figure 2 shows an overview of our framework deployment. Partnering with Merit, we instantiate a filter on a monitoring station overseeing mirrored traffic from a router that handles 20% of the ISP's traffic. The filter performs passive fingerprinting over raw packets, exploiting traffic features unique to OpenVPN. IP and port information of flows flagged by the filter are forwarded to a probing system and then distributed to dedicated probers. The probers send a set of predefined probes specifically designed to fingerprint an OpenVPN server. Finally, probed servers that are confirmed as OpenVPN are logged for manual analysis. Such a two-phase framework

Figure 2. Framework Deployment on Merit Steps: (1) Client connects to VPN servers. (2) VPN connections, along with passing traffic, are being mirrored to the filter. (3) Filter forwards server IP of suspected connections to probers. (4) Targets are sent to each dedicated probers. (5) Probers send probes asynchronously. (6) Connections confirmed by probing are logged.

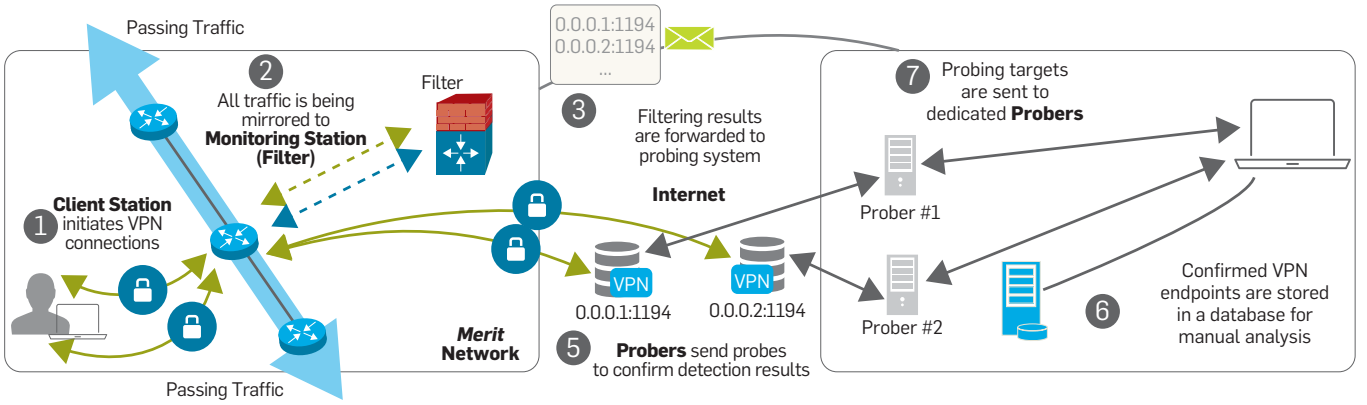
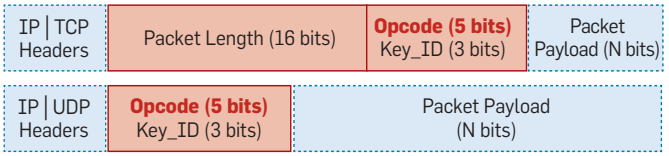


Figure 3. OpenVPN Header in TCP and UDP modes.



resembles how real-world censors operate: lightweight filtering followed up by more expensive, but also more accurate, active probing. This framework is capable of processing massive traffic in real time while also preventing excessive collateral damage.

5. IDENTIFYING FINGERPRINTABLE FEATURES

We identify three OpenVPN fingerprints, exploiting byte pattern, packet length, and server behaviors, respectively.

5.1. Opcode-based fingerprinting.

Each OpenVPN header (Figure 3) starts with an opcode that specifies the message type of the current packet. The opcode field can take more than 10 defined values, corresponding to message types transmitted during different communication stages. A typical OpenVPN session starts with the client sending a `Client Reset`. The server then responds with a `Server Reset`, and a TLS handshake follows. OpenVPN packets that carry TLS ciphertexts have `P-Control` as their message type. Since OpenVPN can run over UDP but has to provide a reliable channel for TLS, each `P-Control` packet is explicitly acknowledged by `P-ACK` packets. Finally, actual payloads are transmitted as `P-Data` packets. Figure 1 illustrates this packet exchange with opcode annotations.

Algorithm 1. Opcode Fingerprinting Logic.

```
Require: N ≥ 0
    OCSet ← {}, CR ← Opcode[0], SR ← Opcode[1]
    i ← 2
    while i ≠ N & i < |Opcode| do
        if Opcode[i] ∈ CR,SR |OCSet| ≥ 4 then
            Return False
        end if
        OCSet += Opcode[i]
        i ← i+1
    endwhile
    Return i == N & 4 ≤ |OCSet| ≤ 10
    #At least 4 different Opcodes needed to complete handshake.
    In total 10 Opcodes defined by the protocol.
```

A packet field taking a fixed number of values can be easy to fingerprint and has been exploited before against other protocols.[2] We fingerprint OpenVPN's handshake sequence by analyzing each opcode byte for the first N packets of a flow. Algorithm 1 shows the process of opcode fingerprinting, with `Opcode` referring to the sequence of N opcode values found in the first N packets of a given flow. Briefly, the filter flags a flow if the number of different opcodes observed accords with the protocol *and* the `Client` and `Server Resets` are not seen once the handshake is completed.

5.2. ACK-based fingerprinting.

OpenVPN engages in a TLS-style handshake with its peer over the control channel. Since TLS is designed to operate over a reliable layer, OpenVPN implements an explicit acknowledgement and re-transmission mechanism for its control channel messages. Specifically, incoming `P-Control` packets are acknowledged by `P-ACK` packets, which do not carry any TLS payloads and are uniform in size (Note: These ACK packets are carried over by TCP as payload and are not the same as TCP ACK flags). These ACK packets are seen mostly only in the early stage of a flow, during the handshake phase, and are not used in the actual data transfer channel, which can run over an unreliable layer.

Previously, the unique timing pattern in meek's TCP-level ACK traffic has rendered the obfuscation tool vulnerable to detection.[21] For OpenVPN, the presence of explicit ACK packets, uniform in size and only seen in some parts of a session, provides another fingerprintable feature. We first identify a likely ACK packet of a session by locating an initial packet exchange sequence of C->S (Client-Reset), S->C (Server-Reset), C->S (ACK), C->S (Control), as illustrated in Figure 1. For vanilla OpenVPN and XOR-based obfus-

cation, the first ACK packet usually appears as the third (data) packet transmitted in a session. For tunnels or obfuscators that have their own handshake or key exchange process (for example, Stunnel, SSH tunnel, or Obfsproxy), this counting is offset by the number of tunnel handshake packets. Next, we group packets into 10-packet bins, and we derive the ACK fingerprint for each flow by counting the number of packets in each bin that have the same size as the identified ACK packet. For OpenVPN flows, we expect to observe a high number of ACK packets in early bins and an absence of them in later bins. This approach proves effective to fingerprint vanilla OpenVPN as well as obfuscated services running over encrypted tunnels that lack random padding.

5.3. Active server fingerprinting.

We explore the feasibility of identifying an OpenVPN server through active probing. Typically, OpenVPN servers respond to a client reset with an explicit server reset, thereby giving away their identity. However, most commercial providers now have adopted *tls-auth* or *tls-crypt* options. These options add an additional HMAC signature—signed by a pre-shared key—to every control channel packet for integrity verification, including the initial reset packets. With either of these options enabled, an OpenVPN server would not respond to an unauthenticated client reset with a server reset, but would instead drop such packets without further processing. The presence of such HMAC mechanism increases the complexity of doing active probing: It effectively makes OpenVPN servers "probe-resistant"[5] by remaining silent when probed by an unauthenticated client.

In fact, similar HMAC mechanisms are used by more popular "probe-resistant" proxies, such as obfs4. However, unlike obfs4 which waits for a server-specific random delay before dropping an unauthenticated connection, OpenVPN always *immediately* closes the connection if a valid HMAC cannot be located. We design our probes to leverage this protocol-specific behavior, and as a result, we manage to fingerprint OpenVPN servers even if they do not respond throughout our probing cycles. The key concept is that although the application may not respond to probing, an attacker may still be able to fingerprint application-specific thresholds at the TCP level, such as timeouts.

We use two datasets in this section to help with designing probes. **ZMap Set**: To construct a realistic non-VPN endpoints dataset, we use ZMap to scan each of the 65,535 TCP ports over the entire IPv4 space, limiting results for each port to 200 endpoints (with the specific port open), resulting in more than 13 million endpoints. **Censys Set**: We query the `Censys.io` database for hosts with TCP port 1194/OpenVPN open. Next, we probe each endpoint with a typical OpenVPN `Client Reset` and group endpoints that respond with explicit `Server Resets`. This results in 180,858 hosts known to be OpenVPN endpoints (with "tls-auth" disabled).

5.3.1. Base probes.

We design probes exploiting a behavior associated with how OpenVPN packetizes TCP streams. When OpenVPN operates over TCP, it needs to split the continuous stream into discrete OpenVPN packets. Figure 4 presents a high-level abstraction of this process. The most relevant parts are: A buffer is allocated in memory to reassemble fragments of OpenVPN packets encapsulated in TCP streams. The length N for the next OpenVPN packet is extracted from the first two bytes of the header, and the routine keeps reading N additional bytes before it returns the reassembled packet to the caller. This means that an OpenVPN packet will not be parsed and checked for syntax and encryption errors until all its parts arrive at the server. Based on this behavior, we design two sequential probes to trigger an OpenVPN server into different code paths—which result in different connection timeouts—and measure the time elapsed before the server responds or terminates the connection. As shown in Table 1, *Base Probe 1* carries a typical 16-byte OpenVPN `Client Reset`, while *Base Probe 2* has the same payload with the last byte stripped off. The assumption is since our two probes only differ in one byte, most non-OpenVPN servers will respond to our probes in a similar way. However, for an OpenVPN server with HMAC enabled, the connection sending the first probe will be dropped immediately because the OpenVPN packet is reassembled and a valid HMAC cannot be located. The second probe will not receive an immediate response, as the server will wait for an additional byte to arrive for reassembly. The connection will stay idle until a server-specific handshake timeout has passed, after which the connection will be dropped. As such, the first probe will be dropped

Figure 4. OpenVPN TCP new packet-processing routines.

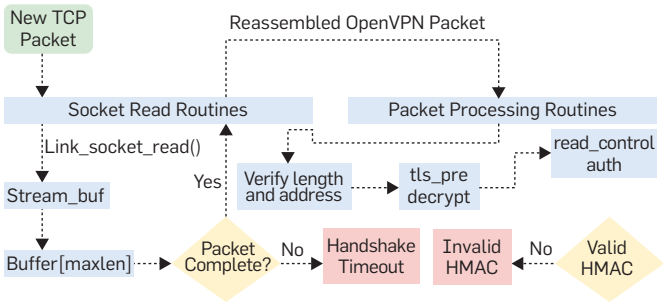
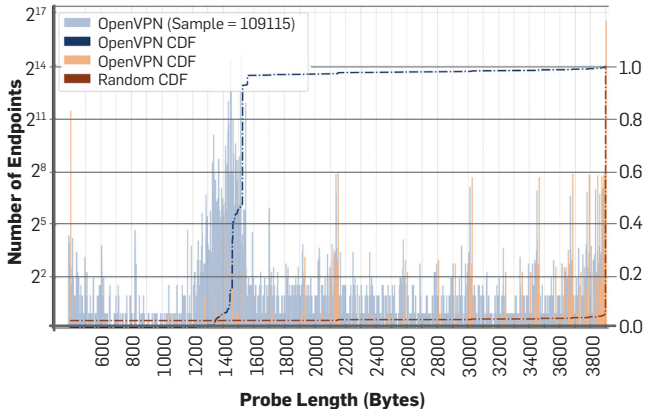


Table 1. Summary of probes and the expected behaviors from an OpenVPN server.

ProbeName	Probe Content	Expect Behavior
BaseProbe 1	x00x0ex38.{8}x00x00x00x00x00	Explicit Reset or Short Close
BaseProbe 2	x00x0ex38.{8}x00x00x00x00	Long Close
Generic	x0dx0ax0dx0a	Short Close
One Zero	x00	Long Close
Two Zero	x00x00	Short Close
Epmd	x00x01x6e	Short Close
SSH	SSH-2.0-OpenSSH-8.1/r/n	Short Close
HTTP-GET	GET/HTTP/1.0 /r /n /r /n	Short Close
TLS	Client Hello by Chromium	Short Close
2K-Random	Random 2000 Bytes	Close & RST

Figure 5. RST thresholds for OpenVPN and random endpoints.



at the decryption routine, while the second probe will be dropped at the packet reassembly routine, both labeled red in Figure 4.

5.3.2. Additional probes.

The two probes, although useful, are limited and there may be other protocols with behaviors similar to OpenVPN. After using both to probe the *ZMap Set*, we still identify a handful of services that respond similarly to OpenVPN servers.

We design additional probes based on the fact that OpenVPN validates packet length and will drop connections sending invalid length without waiting for the next packet to be reassembled. Here, packet length refers to the length declared by the first two bytes of an OpenVPN header (see Figure 3), rather than the TCP packet length. A "valid" length is in the range of [1, *maxlen*], where *maxlen* is derived from the server's MTU configurations. For instance, default TUN MTU of 1,500 bytes, combined with overheads, results in a *max-len* of 1,627 bytes. In this case, probes whose first two bytes have a decimal value greater than 1627 (0x06,0x5B) will be dropped immediately.

We also design probes leveraging the way a Linux server closes a TCP connection. When a TCP connection terminates, the operating systems at both ends typically complete a FIN four-way handshake. However, previous work has found that if a connection is closed with unread bytes in buffer, Linux will send a RST packet.[5] A server's "RST Threshold" is defined as the minimum number of bytes needed to send to the server to trigger a RST. We determine the RST threshold distribution for both *ZMap Set* and *Censys Set*. As shown in Figure 5, the vast majority of OpenVPN servers have a RST threshold around 1550–1660 bytes, corresponding to buffers allocated with typical MTU configurations. In contrast, more than 97% of random ZMap endpoints have a RST threshold less than 500 or greater than 4,000. We therefore construct an additional probe with 2,000 random bytes, which we expect more than 98% of legitimate OpenVPN servers and less than 3% of random servers to respond to with RST packets. Table 1 lists all probes and the expected behaviors from an OpenVPN server.

5.4. Constructing filters and probers.

Our filter performs both opcode and ACK-based fingerprinting, flagging a flow if at least one fingerprint matches. This is because the opcode and ACK fingerprints are designed to be complementary: The former works against XOR-based obfuscations that work like Vigenère ciphers; the latter targets tunneling-based obfuscation that lacks random padding and preserves the 1:1 correspondence between the original and obfuscated packet streams. Combining the two features maximizes our fingerprinting coverage. Following filter's result, the prober performs active probing to further lower potential false positives.

We implement the filter in Zeek. We note that the evaluation processes for opcode and ACK-based fingerprinting are quite simple: Both only require several dozen integer comparisons, limited by the observation window, while maintaining a small number of per-flow states. We implement the prober in Nim. We believe that both components can be easily deployed by any ISP or censor.

6. ADDITIONAL PRACTICAL CONSIDERATIONS

In this section, we discuss how we fine-tuned our system for deployment as well as several practical considerations that can affect the system performance, such as packet loss or port sharing:

- **Quantifying detection thresholds**: Includes determining the exact ACK fingerprints, a sequence of thresholds derived from a decision-tree model trained on datasets with OpenVPN and non-VPN traffic.
- **Windowing strategy**: Considers the trade-offs between detection speed and accuracy, as well as the use of windowing strategies to limit inspection.
- **Effects of packet loss**: Examines how packet loss affects the performance of the filter, including simulation of random packet loss.
- **Server churn for asynchronous probing**: Explores synchronous versus asynchronous probing to balance efficiency and potential IP churn.
- **Probe UDP and obfuscated OpenVPN servers**: Discusses a strategy for probing adjacent netblocks of suspected UDP or obfuscated endpoints to reveal nearby vanilla TCP servers.
- **Complication from port sharing**: Accounts for the situation where OpenVPN shares the listening port with another application.
- **Ethics, privacy, and responsible disclosure**: Discusses how we consider the security and privacy risks and ethical issues raised by our work and the procedural and technical steps we take to mitigate the risks.

Due to space constraints, a detailed exploration of these aspects is not provided in this section; for the full discussion, please refer to the USENIX version of the paper.

7. REAL-WORLD DEPLOYMENT SETUP

We set out to explore if an ISP or censor can fingerprint OpenVPN connections at scale, without significant collateral damage. Adopting the viewpoint of an adversarial ISP, we deploy our framework inside Merit, as shown in Figure 2. Our evaluation is twofold: We generate *control* vanilla

and obfuscated flows with commercial VPN providers and attempt to identify them as a network intermediary; we also process other traffic passing through our Monitoring Station in order to estimate the false positive rate of our framework.

We set up our filters on a 16-core server (monitoring station) inside Merit with two mirroring interfaces that have an aggregated 20Gbps bandwidth. We set up *probers* on two dedicated measurement machines, each provisioned with 10 IPv4 and 1 IPv6 addresses. By the end of each day during the evaluation, the probers fetch filtering logs from the monitoring station. For each target, we run a Masscan to the /29 subnet the IP belongs to over all TCP ports (1-65535). We follow up each discovered open port by running our probing scheme, and endpoints confirmed through probing are recorded for manual analysis.

To select VPN services for evaluation, we first generate a list of "top" VPN services ranked by popularity. We combine 80 providers, most of which are paid premium VPN services, from top VPN recommendation sites based on previous work.[11] Next, we visit the websites of these VPN providers searching for "Obfuscation," "Stealth," or "Camouflage Mode," among others, and include providers that offer at least one obfuscated VPN configuration. In total, we find 24 providers offering obfuscated services. We test all obfuscation configurations if more than one is offered as well as vanilla OpenVPN for each provider. If TCP and UDP modes are both available, we test them separately. In total, we have 81 configurations, 41 of which are obfuscated ones.

We configure the Client Station inside Merit to act as a VPN client. Both upstream and downstream traffic of the Client Station go through the router that mirrors traffic to the monitoring station. In addition, we exclude this server from our random sampling so that *all* traffic to/from this server will be analyzed. On the client, we run an automated script to generate *control* traffic for our evaluation. For each VPN configuration, the process was repeated 50 times and packet captures were collected for reference.

8. EVALUATION AND FINDINGS

Overall, we are able to identify 1,718 out of 2,000 vanilla flows, corresponding to 39 out of 40 unique configurations. This suggests the majority of OpenVPN traffic and servers are vulnerable to passive filtering and active probing, respectively. The few exceptions correspond to VPN providers that only offer UDP-based services or hide their servers behind IDS, which thwarts our probing attempts. Surprisingly, we also identify more than two-thirds of all obfuscated flows, corresponding to 34 out of 41 obfuscated configurations. This result is mostly due to obfuscated services using OpenVPN as their backbone protocol and insufficient obfuscation failing to mask OpenVPN's fingerprints. Alarmingly, out of the "top 10" VPN providers ranked by *top10vpn.com*, eight provide obfuscation services of some sort, suggesting that being undetectable is within the providers' threat model for their clients. Yet, *all* of them are flagged as suspect flows due to either insufficient encryption (Opcode) or insufficient obfuscation over packet length (ACK). Considering that these obfuscated VPN services usually claim to be "undetectable" or claim that the obfuscation "keeps you out of trouble", this result is alarming as users who use these services may have a false sense of privacy and "unobservability."

Detailed results with breakdown by each *control* VPN configuration can be found in the full version of the paper.

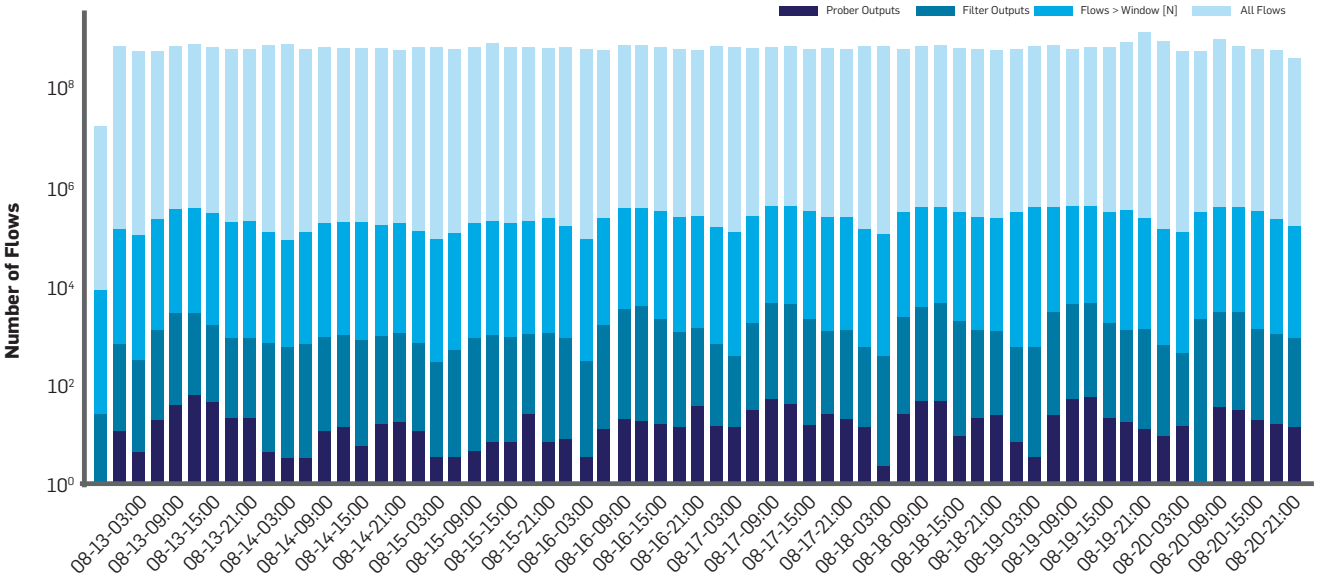
Four out of the top five VPN providers use XOR-based obfuscation, which is easily fingerprintable. We find that among the top five VPN providers, four offer obfuscated services, all of which nonetheless are flagged as OpenVPN flows by our filter more than 90% of the time. A closer look at the raw packet capture suggests that *all* of them employ obfuscations that are almost identical to the unofficial XOR patch. Although the patch can bypass some of the most basic filters adopted by existing open source DPI tools, we have demonstrated that even a slightly more sophisticated filter will be able to reliably and accurately detect them.

Wrapping OpenVPN inside encrypted tunnels is a popular obfuscation strategy, yet some flows are still recognizable due to a lack of random padding. Another popular class of obfuscation strategies is tunnel-based, which wraps OpenVPN traffic inside an encrypted tunnel to frustrate any analysis over packet payloads. Examples include Stunnel, SSH tunnel, Shadowsocks, obfs{2/3/4}, and V2Ray(VMess). Overall, we find 20 obfuscated configurations deployed by 14 VPN providers that are tunnel-based. However, most of these tunnels do not add random padding to the payload being tunneled, with the only exceptions being obfs4 and VMess, which can draw packet sizes from certain distributions. Among the 20 tunnel-based obfuscated services, only three of them deploy obfs4 and only one deploys VMess, leaving the remaining 16 vulnerable to ACK fingerprinting. We note that this does not mean these tunneling tools do not work, but rather that protection against traffic analysis is not among the design goals.

UDP and obfuscated servers often share infrastructure with vanilla TCP servers, leaving them "guilty by association". We discover that the majority of UDP and obfuscated OpenVPN services are co-located with vanilla TCP servers. For example, TorGuard hosts vanilla and Stunnel-obfuscated OpenVPN instances on the same host but different ports, whereas Perfect Privacy hosts them in neighboring IPs (*.*.193.26 for vanilla, *.*.193.27 for Stunnel, *.*.193.28 for SSH, and *.*.193.29 for obfs3). We find that for 34 out of 41 obfuscated services, at least one vanilla OpenVPN TCP server can be found within the server's /29 subnet. Similarly, we were able to actively probe 18 out of 20 UDP configurations due to their co-location with TCP servers. In addition, we also find five providers sharing infrastructures used by their obfuscated services. This result is only a lower bound as we did not connect to every single server available from each provider. Obfuscated services using shared infrastructures may be easier for adversaries to identify and block.

On the positive side, some deployed services successfully evade our detection. Some providers deploy randomizers such as obfs4, v2ray, or proprietary protocols with

Figure 6. Merit evaluation results over days, excluding Control VPN flows.



random padding, which stopped us at the filtering stage (for example, Tunnelbear). In addition, some providers deploy their obfuscated servers behind a firewall or IDS, or do not host vanilla OpenVPN TCP servers at all, such as VyprVPN, which currently only supports UDP as transport. For these providers, even though our filter flags their flows as suspected OpenVPN, we were not able to confirm with subsequent probing.

Characterizing false positives. Figure 6 shows an hour-level breakdown of the evaluation statistics, excluding *control* flows. Overall, both the filter and prober are able to reduce the number of suspected flows by several orders of magnitude, which when combined flagged 3,638 flows as OpenVPN connections. Among these flows, the destination servers for 469 of them respond to our *Base Probe* with an explicit server reset, indicating the presence of a legitimate OpenVPN server not configured with HMAC protection. For the remaining 3,169 flows, we attempt to further characterize them based on circumstantial evidence about the destination endpoint, such as their co-location with TLS, WHOIS Record, ISP name, IP context, and so on.

Overall, our seven-day evaluation flagged 3,638 flows identified as "OpenVPN" from more than 10 million flows that exceed our observation window. Among these, we can find evidence supporting our detection result for 3,245 flows. Most of the remaining 393 flows have server IPs belonging to cloud hosting services, and we are not able to further classify them. Conservatively, we can upper bound the false-positive rate to 0.0039%—three orders of magnitude lower than previous ML-based approaches (1.4%-5.5%).[3,6]

9. DISCUSSION AND MITIGATION

ISPs and censors are motivated to detect VPN flows in order to enforce traffic policies and information controls. We demonstrate that tracking and blocking the use of OpenVPN, even with most deployed obfuscation methods, is practical at scale and with minimal collateral damage. We note that many VPN providers' claims that their obfuscated services are unobservable appear to be misleading and potentially dangerous, especially to users from countries where personal VPN usage is illegal. In light of our findings, users should *not* expect complete unobservability even when connected to "obfuscated" OpenVPN-based services.

Putting the human danger aside, the ease of fingerprinting makes OpenVPN more susceptible to throttling or blocking from ISPs and governments. Previous research suggests that some censors already use two-stage pipelines, which are highly similar to our deployment, to detect other protocols such as Tor or Shadowsocks.[2,23] These adversaries can quickly adapt such infrastructure to detect OpenVPN traffic by simply adding protocol-specific fingerprints and probes. Furthermore, while we focus on OpenVPN due to its overwhelming popularity among commercial VPNs, it is possible to extend our two-stage framework to other VPN protocols (for example, WireGuard and StrongSwan) by analyzing their traffic patterns and server behaviors. Censors can also quickly adopt these fingerprints to track and block VPN usage during sensitive times, such as political upheavals, when VPN connections are most vital to the free flow of information.

There are several defensive strategies to achieve near-term protection from the fingerprinting attacks we describe. First, VPN providers offering both vanilla and obfuscated OpenVPN services should avoid co-locating them. Ideally, obfuscation servers should be well separated from OpenVPN instances in the network address space and operate as "bridge servers" that forward client traffic to VPN servers elsewhere. Second, VPN providers should switch from static to random padding for their obfuscated services. As we have shown, for protocols with a distinctive handshake phase, even the most basic threshold-based detector is able to fingerprint them by packet sizes. Third,

we suggest that the OpenVPN developers follow recommendations from previous work with regard to how servers respond to failed handshake attempts. Servers closing failed connections immediately or in a predictable manner has enabled active probing attacks against a variety of other protocols.[2,5] In response, these protocols have implemented either unlimited timeouts (reading from the buffer indefinitely) or diversified close behaviors (in which each server instance closes failed connections in a different manner).

In the long term, we fear that the cat-and-mouse game between censors and circumvention tools, such as the Great Firewall and Tor, will occur in the VPN ecosystem as well, and developers and providers will have to adapt their obfuscation strategies to the evolving adversaries. We urge commercial VPN providers to adopt more standardized obfuscation solutions, such as Pluggable Transports,[10] and to be more transparent about the techniques used by their obfuscated services. This transparency will help foster development of stronger obfuscation methods and encourage developers to design better techniques to overcome the progress of information control technologies. Additional future work is needed to characterize the performance costs of different approaches to VPN obfuscation and to help users with varying threat models make appropriate trade-offs between performance and resilient unobservability.

10. CONCLUSION

We demonstrate that OpenVPN, even with widely applied obfuscation techniques, can be reliably detected and blocked at scale by network-based adversaries. Inspired by previous real-world censorship events, we designed a two-phase system that performs passive filtering followed by active probing to fingerprint OpenVPN flows. We evaluated the practicality of our approach in partnership with a mid-size ISP, and we were able to identify the majority of vanilla and obfuscated OpenVPN flows with only negligible false positives.

Users worldwide rely on VPNs to protect their security and privacy and to escape Internet censorship, yet the ease of fingerprinting OpenVPN traffic and the commodification of DPI technologies bring monitoring and blocking of popular VPN services within reach for almost any network operator. We propose several short-term mitigations that can help defend against these threats, but in the long term, we urge VPN providers to adopt more resilient and better standardized obfuscation approaches.

Acknowledgment

This material is based upon work supported by the National Science Foundation under Grant No.1518888, 1823192, 2007741, 2042795, 2120400.

References

1. Openvpn xor patch; https://github.com/clayface/openvpnxorpatch.
2. Alice, B., Carol, J.B., and Houmansadr, A. How china detects and blocks shadowsocks. In *ACM Internet Measurement Conf. (*2020).
3. Bagui, S. et al. Comparison of machine-learning algorithms for classification of vpn network traffic flow using time-related features. In *J. Of Cyber Security Technology* (2017).
4. Ensafi, R. et al. Examining how the great firewall discovers hidden circumvention servers. In *Proceedings of the 2015 Internet Measurement Conf.*
5. Frolov, S., Wampler, J., and Wustrow, E. Detecting probe-resistant proxies. *Network and Distributed System Security* (2020).
6. Fu, P. et al. A netflow sequence attention network for virtual private network traffic detection. In *Proceedings of the Intern. Conf. On Web Information Systems Engineering.*
7. Gil, G., Lashkari, A., Mamun, M., and Ghorbani, A. Characterization of encrypted and VPN traffic using time-related features. In *Proceedings of the 2nd Intern. Conf. on Information Systems Security and Privacy (*2016).
8. Hoogstraaten, H. *Evaluating Server-Side Internet Proxy Detection Methods (Msc Thesis)*, 2018.
9. Houmansadr, A., Brubaker, C., and Shmatikov, V. The parrot is dead: Observing unobservable network communications. *2013 IEEE S&P.*
10. Pluggable transports; https://pluggabletransports.info/.
11. Ramesh, R., Evdokimov, L., Xue, D., and Ensafi, R. Vpnalyzer: Systematic investigation of the VPN ecosystem. *Network and Distributed System Security* (2022).
12. Ramesh, R. et al. Decentralized control: A case study of russia. *Network and Distributed System Security* (2020).
13. *Attention to Companies Using VPN Services in Operation*; https://rkn.gov.ru/news/rsoc/news73628.htm.
14. Sundara Raman, R. et al. Measuring the deployment of network censorship filters at global scale. *Network and Distributed System Security* (2020).
15. Tolley, W. et al. Blind in/on-path attacks and applications to VPNs. In *Proceedings of the 30th USENIX Security Symp. (2021).*
16. Tschantz, M., Anonymous, S.A., and Paxson, V. Sok: Towards grounding censorship circumvention in empiricism. In *Proceedings of the 2016 IEEE Symp. On Security and Privacy.*
17. Khanna, M. Indian Parliamentary Committee wants to ban VPN services in India. *India Times*; https://bit.ly/4eDsVTX.
18. Chen, C. Chinese government orders ISPs to block personal VPN use. *Private Internet Access*; https://bit.ly/4fXbIpl/.
19. China's firewall technology upgrades virtual private network management and control tightening. *RFA*; https://www.rfa.org/mandarin/yataibaodao/cyl-12212012155229.html.
20. *McKane, J.* Rain throttles Internet speeds for customers on VPNs. *My Broadband (2021)*; https://bit.ly/4hT841N.
21. Wang, L. et al. Seeing through network-protocol obfuscation. In *Proceedings of the 22nd ACM SIGSAC Conf. on Computer and Communications Security* (2015).
22. Webb, A. and Reddy, A. Finding proxy users at the service using anomaly detection. In Proceedings of the *2016 IEEE Conf. On Communications and Network Security (CNS).*
23. Winter, P. and Lindskog, S. How the great firewall of China is blocking Tor. In *Proceedings of the 2nd USENIX Workshop on Free and Open Communications on the Internet (FOCI 12)*. USENIX Assoc.

Diwen Xue (diwenx@umich.edu), University of Michigan, Ann Arbor, MI, USA.

Reethika Ramesh (reethika@umich.edu), University of Michigan, Ann Arbor, MI, USA.

Arham Jain (arham@umich.edu), University of Michigan, Ann Arbor, MI, USA.

Michalis Kallitsis (mgkallit@merit.edu) Merit Network, Inc., Ann Arbor, MI, USA.

J. Alex Halderman (jhalderm@umich.edu), University of Michigan, Ann Arbor, MI, USA.

Jedidiah R. Crandall (jedimaestro@asu.edu), Arizona State University, Tempe, AZ, USA.

Roya Ensafi (ensafi@umich.edu), University of Michigan, Ann Arbor, MI, USA.

Copyright © 2025 held by the owner/author(s). Publication rights licensed to ACM.